CS3400+CS3410 — Principles of Software Engineering (+ lab)

Tier	B
Phase (order)	5
Do this first	CS2705, CS1234
Estimated effort	8-10 weeks
Unlocks	Any large project

Roadmap

1. Process models, requirements, UML/modelling.
2. Design principles, patterns, refactoring.
3. Testing (unit, integration), debugging, CI.
4. Estimation, QA, configuration management.

Video courses (all links checked on YouTube on 2026-10-22)

🎓 Main course: follow it start to end

- Software Engineering by Prof. Rajib Mall (60 videos). NPTEL *Software Engineering* (Rajib Mall, IIT Kharagpur): the standard Indian SE course, matches the IITM syllabus.

🎥 Other full lecture series (pick one if the main one doesn't click, or for topics it skips)

- Software testing (59 videos). NPTEL-NOC IITM *Software Testing* (Meenakshi D'Souza, IIIT Bangalore): for the testing half.
- Object-Oriented Analysis and Design (52 videos). *Object-Oriented Analysis and Design* (NPTEL, IIT Kharagpur): UML and design patterns.

⚡ Short videos for intuition and revision

- Missing Semester IAP 2020 (11 videos). Missing Semester lectures 6-8 (git, debugging, build systems/CI) for the lab.

Study plan (week by week)

Lecture numbers are positions in the main playlist above. Click a number to open that video.

Week	Lectures	What to do
1	1–14	Intro, life-cycle models, waterfall, incremental, agile, XP, Scrum.

Week	Lectures	What to do
2	15–18	Requirements: gathering, SRS. Write an SRS for your project.
3	19–30	Design: cohesion, coupling, DFDs, structured design.
4-5	31–42	UML: use cases, class, sequence, state diagrams; OO design process. Draw UML for your project; read *Head First Design Patterns*.
6-7	43–60	Testing: unit, equivalence class, boundary, decision table, pairwise, white-box, MC/DC, path, dataflow and mutation testing. Write a pytest suite with coverage >80% and try mutmut.

▼ **All lectures in *Software Engineering by Prof. Rajib Mall* (60 videos)**

1. Lecture 01: Introduction- I
2. Lecture 02: Introduction- II
3. Lecture 03: Introduction- III
4. Lecture 04: Introduction- IV
5. Lecture 05: Introduction- V
6. Lecture 06: Life Cycle Model
7. Lecture 07: Life Cycle Model (Contd.)
8. Lecture 08: Waterfall Model
9. Lecture 09: Waterfall Derivatives
10. Lecture 10: Incremental Model
11. Lecture 11: Evolutionary Model
12. Lecture 12: Agile Model
13. Lecture 13: Extreme Programming and Scrum
14. Lecture 14: Scrum
15. Lecture 15: Introduction to requirement specification
16. Lecture 16: Requirement gathering and analysis
17. Lecture 17: Functional requirements
18. Lecture 18: Representation of complex programming logic
19. Lecture 19: Design Fundamentals
20. Lecture 20: Modular Design
21. Lecture 21: Classification of Cohesion
22. Lecture 22: Classification of Coupling
23. Lecture 23: Introduction to structured analysis and structured design
24. Lecture 24: Basics of Data Flow Diagrams (DFD)
25. Lecture 25: Developing DFD Model
26. Lecture 26: Examples of DFD Model development
27. Lecture 27: DFD Model - More Examples
28. Lecture 28: Essentials of Structure Chart
29. Lecture 29: Structure Chart Development
30. Lecture 30: Structured Design Examples

31. Lecture 31: Use Case Modelling
32. Lecture 32: Factoring Use Cases
33. Lecture 33: Overview of Class diagram
34. Lecture 34: Inheritance relationship
35. Lecture 35: Association relationship
36. Lecture 36: Aggregation/ Composition and dependency relations
37. Lecture 37: Interation Modelling
38. Lecture 38: Development of Sequence diagrams
39. Lecture 39: State-Machine diagram
40. Lecture 40: An Object-Oriented design process
41. Lecture 41: Domain Analysis
42. Lecture 42: Examples of object-oriented design
43. Lecture 43: Basic concepts in Testing-I
44. Lecture 44: Basic concepts in Testing-II
45. Lecture 45: Basic concepts in Testing-III
46. Lecture 46: Unit testing strategies-I
47. Lecture 47: Unit testing strategies-II
48. Lecture 48: Equivalance Class Testing-I
49. Lecture 49: Equivalance Class Testing-II
50. Lecture 50: Special Value Testing
51. Lecture 51: Combinatorial Testing
52. Lecture 52: Decision Table Testing
53. Lecture 53: Cause effect graphing
54. Lecture 54: Pairwise Testing
55. Lecture 55: White box Testing
56. Lecture 56: Condition Testing
57. Lecture 57: MC/DC Coverage
58. Lecture 58: MC/DC Testing
59. Lecture 59: Path Testing
60. Lecture 60: Dataflow and Mutation Testing

Books, notes and websites

- MIT 6.031 *Software Construction* notes
- Book: *A Philosophy of Software Design* (Ousterhout)
- Book: *Design Patterns* (Gamma et al.)
- Pro Git; GitHub Actions docs

Practice / proof of understanding

Build a medium app (about 2k lines) with tests, CI, issues and a design doc.

What your IITM course sheet says

From the PDF: CS3400

Official description: The process of software development - Process models. Software measurement - Software process improvement by metrics- Size-oriented metrics and function point metrics. Engineering of software systems - Analysis concepts and principles - Analysis modeling - Data modeling - Specification techniques for software. Design concepts and principles - Abstraction - Refinement - Modularity - Architectural design - Cohesion coupling concepts - Refactoring of designs. O-O Approach to software design - Concepts - Design issues - Modeling techniques Design process - Design patterns. Software Testing - Principles - Designing test cases - Testing strategies - Debugging - Introduction to O-O testing. Software project planning - Effort and cost estimation techniques - LOC-based and Function-point based measures - The COCOMO model. Software Quality Assurance(SQA) - Software Reliability - The IS

Official textbooks: NIL

Official references: 1. Software Engineering - A Practitioner's approach by Roger Pressman, 5e, 2001. 2. Software Engineering Economics by Barry Boehm. 3. Fundamentals of Software Engineering by Carlo Ghezzi, Mehdi Jazayeri, Dino Mandrioli. 4. IEEE Software Engineering Standards. 5. "Software Metrics : A rigorous and practical approach", 2nd edition, Norman E Fenton and Shari Lawrence Pfleeger, Thomson Asia, 2002

From the PDF: CS3410

Official description: System Requirement Specification (SRS) and related analysis documents as per the guidelines in ANSI/IEEE Std 830-1984. Design documents representing the complete design of the software system. The design can further be improved by the principles learned in this module. Analysis and design for the same problem should be done using O-O approach. Test documents as per ANSI/IEEE Std. 829/1983 Software Test Documentation. Simple exercises in effort and cost estimation in COCOMO model. Application of COCOMO and Function Point (FP) model for the actual project that has been chosen. Familiarization of SCM tools with some public domain software like SCCS, CVS. Familiarization of some reverse engineering tools available in the public domain. At the end of the semester, there should be a presentation of the project with demonstration. It is also advisable to have the students present the

Official textbooks: NIL